

Arquitectura de Computadoras (72.08)

Informe Trabajo Práctico Especial



Grupo 5

Integrantes:

- Fraga, Santiago - 64346
- Goldbaum, Carolina - 65112
- Choe, Ivonne - 64880

Fecha de entrega: 5 de noviembre de 2025

Índice:

Introducción.....	3
Kernel/Userland.....	3
Interrupciones.....	4
Excepciones.....	4
System calls.....	4
Drivers.....	6
Driver de Teclado.....	6
Driver de video.....	6
Shell.....	6
Tron.....	7

Introducción

El presente trabajo práctico especial tiene como objetivo la implementación de un sistema operativo básico capaz de administrar los recursos de hardware de una computadora bajo una arquitectura Intel de 64 bits. A partir del código base provisto por la cátedra, se desarrolló un kernel booteable que interactúa directamente con el hardware mediante drivers y que ofrece una API para que las aplicaciones en user space puedan acceder a sus funcionalidades.

El proyecto busca establecer la correcta separación entre el kernel space, que interactúa directamente con el hardware a través de drivers, y el user space, donde se ejecutan las aplicaciones de usuario. De esta manera, se garantiza que las aplicaciones no accedan directamente al hardware, sino únicamente a través de una API controlada proporcionada por el kernel.

Para demostrar el funcionamiento del sistema, se implementó una shell interactiva que permite ejecutar diversos programas, incluyendo funciones de ayuda, visualización de la hora del sistema, acceso a los registros del procesador y un módulo gráfico que ejecuta el juego "Tron". Además, el kernel incorpora mecanismos de manejo de excepciones (división por cero y operación inválida) que contribuyen al correcto funcionamiento del sistema.

A lo largo del informe se detallan las decisiones de diseño, la estructura del sistema y otros aspectos relevantes que permiten comprender el proceso de desarrollo e implementación del trabajo.

Kernel/Userland

El sistema operativo desarrollado se encuentra dividido en dos espacios principales: kernel space y user space

El kernel se encarga de gestionar los recursos de hardware y brindar un entorno para la ejecución de las aplicaciones de usuario. Entre sus tareas se incluyen el manejo de interrupciones, excepciones, system calls y la administración de drivers que permiten la comunicación con dispositivos como el teclado y la pantalla.

En cuanto al userland, está compuesto por los programas que interactúan con el kernel mediante las syscalls. Estas aplicaciones no acceden directamente al hardware, sino que dependen de la API provista por el kernel. Dentro del userland se encuentran la shell, los programas de prueba para ver el buen funcionamiento de las excepciones y el juego Tron, las cuales demuestran las funcionalidades del sistema.

Cabe destacar que la comunicación entre ambos espacios se realiza a través de la interrupción 0x80, que actúa como punto de entrada al dispatcher de las syscalls. Esto permite que las aplicaciones en el user space soliciten servicios al kernel, como por ejemplo lectura de entrada, escritura en pantalla o acceso a la hora del sistema, sin comprometer la seguridad del sistema operativo.

Interrupciones

Cada interrupción tiene un número asociado y una entrada en la tabla de descriptores de interrupción (IDT), que indica qué rutina debe ejecutarse en cada caso.

Estas rutinas están implementadas en el archivo "interrupts.asm", que se encarga de guardar el estado del procesador, llamar a la función correspondiente en C y luego restaurar el contexto para que el programa pueda continuar normalmente.

Cuando llega una interrupción, se llama a la función "irqDispatcher()", que se encarga de identificar qué interrupción ocurrió y redirigirla al módulo adecuado. En este caso, se tuvieron en cuenta dos interrupciones: la del Timer Tick y la del Teclado.

- Si se trata de la IRQ0 (Timer), se llama a "timer_handler()", que actualiza el contador de ticks del sistema y por ende permite controlar el tiempo o pausar procesos.
- Si se trata de la IRQ1 (Teclado), se llama a "keyboard_handler()", que principalmente se encarga de procesar las teclas presionadas.

Además de las interrupciones de hardware, el sistema implementa la interrupción 0x80 que se utiliza para las syscalls y también se encarga del manejo de las excepciones, las cuales se detallarán más adelante.

Excepciones

En este proyecto se implementaron dos excepciones principales: división por cero (ID 0) y operación inválida (ID 6).

Ambas son atendidas por la función "exceptionDispatcher()", que se encarga de identificar el tipo de excepción y redirigirla a la rutina correspondiente.

Cuando ocurre una excepción, el kernel limpia la pantalla y muestra un mensaje en color rojo con el tipo de error detectado.

Luego se imprimen en pantalla todos los registros del procesador utilizando la función "print_registers_graphic()", lo que permite visualizar el estado exacto del CPU en el momento del error.

Una vez mostrada la información, el sistema espera a que el usuario presione Enter para continuar.

Al hacerlo, se limpia nuevamente la pantalla y se reinicia el flujo de ejecución del Userland, apuntando el registro de instrucción ("RIP") a la dirección de inicio del shell.

De esta manera, el sistema puede recuperarse del error y volver a un estado estable sin necesidad de reiniciarse.

System calls

Todas las llamadas al sistema se realizan mediante la interrupción int 0x80, donde el número de syscall se pasa en el registro RAX. El kernel cuenta con un total

de 14 llamadas al sistema (índices 0–13), implementadas en el archivo `syscall_dispatcher.c`. A continuación se describen cada una de ellas:

0. SYS_READ:

Lee caracteres del teclado (`stdin`) de forma bloqueante. Maneja backspace, enter, tab y caracteres imprimibles.

1. SYS_WRITE:

Escribe texto en pantalla (`stdout`) con manejo automático del cursor. Soporta saltos de línea, backspace y scrolling automático.

2. SYS_CLEAR:

Limpia la pantalla con un color de fondo especificado y resetea el cursor a la posición inicial (10, 10).

3. SYS_DRAW_AT:

Dibuja texto en coordenadas específicas de la pantalla. Útil para UI y juegos.

4. SYS_TIME:

Obtiene la fecha y hora actual empaquetada en un `uint64_t` con formato: [Año|Mes|Día|Hora|Minutos|Segundos].

5. SYS_TICKS:

Retorna el contador de ticks del sistema (contador de interrupciones del timer).

6. SYS_SET_SCALE:

Ajusta la escala/zoom del texto (valores válidos: 1-5). Con `delta=-2` retorna la escala actual sin cambiarla.

7. SYS_DRAW_RECT:

Dibuja un rectángulo relleno en coordenadas específicas con color personalizado.

8. SYS_GET_KEY:

Obtiene una tecla de forma no bloqueante. Retorna el scancode o 0 si no hay tecla disponible.

9. SYS_GET_SCREEN_INFO:

Obtiene la resolución de pantalla. Retorna ancho en los 32 bits superiores y alto en los 32 bits inferiores.

10. SYS_SLEEP:

Suspende la ejecución por un número especificado de tics del sistema.

11. SYS_SPEAKER_PLAY:

Reproduce un sonido en el PC speaker a una frecuencia específica (en Hz, máximo 20000 Hz).

12. SYS_SPEAKER_STOP:

Detiene el sonido del PC speaker.

13. SYS_GET_REGS:

Obtiene el snapshot de los registros de la CPU guardado cuando se presiona ESC. Retorna -1 si no hay snapshot disponible.

Drivers

• Driver de Teclado

El driver de teclado permite detectar y procesar las teclas presionadas por el usuario.

Su funcionamiento se basa en la interrupción de hardware IRQ1, que se activa cada vez que se presiona o suelta una tecla.

Cuando ocurre esta interrupción, el procesador ejecuta el handler definido en interrupts.asm, el cual invoca a la función keyboard_handler() dentro del kernel.

El controlador lee el scancode enviado por el teclado, lo traduce a su carácter correspondiente y lo almacena en un buffer interno.

Luego, mediante las llamadas al sistema sys_read() y sys_get_key(), los programas del espacio de usuario pueden obtener el texto ingresado o las teclas presionadas de manera controlada.

• Driver de video

El driver de video es el encargado de la salida visual del sistema.

Proporciona un conjunto de funciones que permiten dibujar texto, limpiar la pantalla o representar figuras gráficas directamente sobre el framebuffer.

Estas operaciones se realizan mediante las llamadas al sistema sys_write(), sys_clear(), sys_draw_at() y sys_draw_rect(), que encapsulan las funciones de bajo nivel implementadas en videoDriver.c.

El kernel mantiene además un cursor interno para controlar la posición de escritura, así como una escala variable de texto que puede modificarse dinámicamente con sys_set_scale().

Shell

La shell implementada proporciona una interfaz de línea de comandos interactiva mediante una tabla de comandos con punteros a función que mapea nombres a sus ejecutores. El bucle principal imprime un >, lee entrada del usuario

mediante `syscall read()` en un buffer de 256 bytes, y ejecuta el comando correspondiente mediante búsqueda en la tabla hasta la entrada 'enter'

Comandos disponibles:

- **help:** Muestra lista completa de comandos disponibles con sus descripciones.
- **time:** Obtiene y muestra la fecha y hora actual del RTC en formato DD/MM/YYYY HH:MM:SS
- **zoomin / zoomout:** Ajustan dinámicamente la escala del texto en pantalla
- **Regs:** Imprime un snapshot completo de los registros de CPU (RAX-R15, RIP, RFLAGS, CS, SS) capturados mediante la syscall `get_regs()`, útil para debugging.
- **cerodiv:** Ejecuta intencionalmente una división por cero
- **invalido:** Dispara una excepción de opcode inválido (#UD) mediante `run_exception_test_invalid_opcode()` para verificar el manejo de excepciones.
- **Tron:** Inicia el juego TRON en pantalla completa, guardando y restaurando la escala de texto al salir.
- **Canción:** Reproduce la melodía de Super Mario Bros mediante el speakers
- **Loop:** Ejecuta un bucle infinito instrumentado que muestra un contador y permite capturar el estado de registros presionando ESC.
- **Clear:** Limpia completamente la pantalla
- **Exit:** Termina la shell ordenadamente estableciendo la flag `shell_exit = 1` que rompe el bucle principal.

Tron

El juego Tron está implementado como un juego de motos de luz clásico con múltiples modos: un jugador, dos jugadores competitivos, y modo versus IA.

Usando una grilla lógica bidimensional (`grid[][]`) que mapea el estado del tablero, permitiendo rastrear posiciones ocupadas por las estelas de luz. Los jugadores se representan mediante estructuras `Player` que contienen posición (x,y), dirección actual, color y estado de vida.

El motor del juego ejecuta un game loop que procesa entrada del teclado (WASD para jugador 1, IJKL para jugador 2), actualiza posiciones según la dirección actual, verifica colisiones consultando la grilla, y renderiza cada frame dibujando rectángulos de color en las nuevas posiciones.

Incluye características adicionales como un sistema de puntuación persistente entre rondas, contador de FPS en tiempo real calculado mediante ticks del timer, un menú con fondo retro estilo grid de perspectiva, efectos de sonido de muerte (`play_death_sound()`), y una IA oponente implementada mediante algoritmos de pathfinding en `tron_ai.c`. El juego maneja diferentes resoluciones de pantalla dinámicamente y ajusta la escala de texto automáticamente al entrar/salir

Una contra es que el juego avanza a la velocidad mas rapida posible (de a un grid), ya que el actualizar la posición cada 1 timertick

